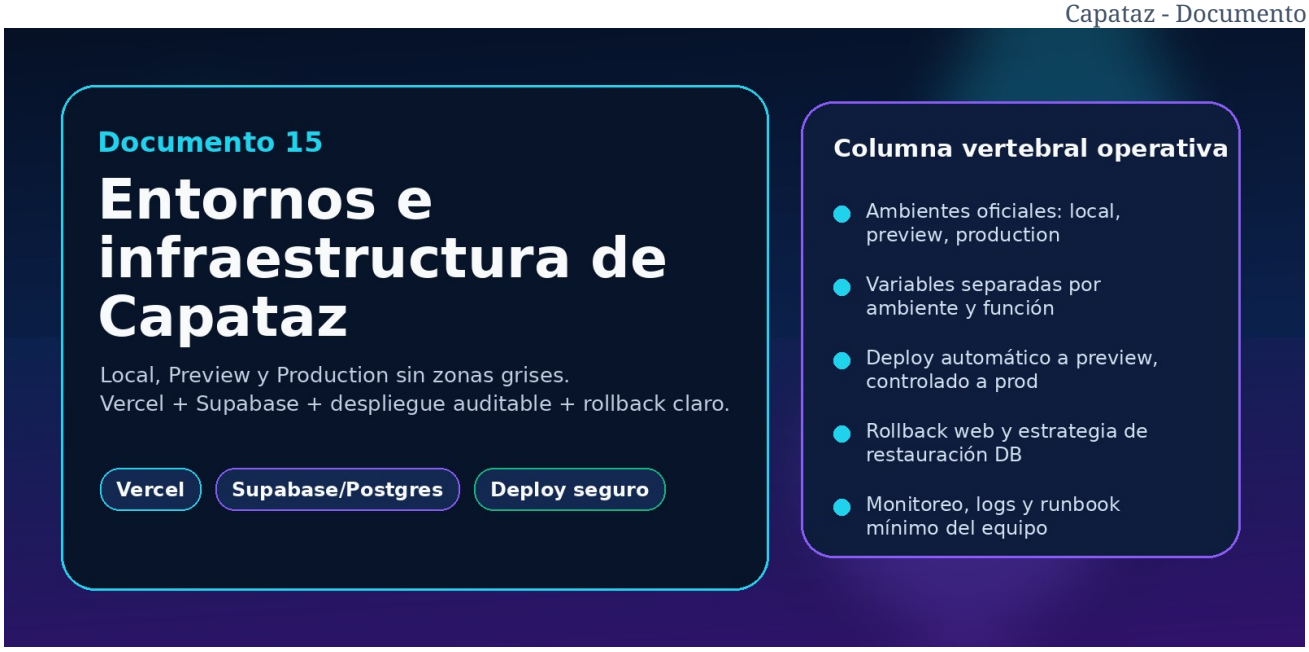

15. Entornos e infraestructura de Capataz

Versión 1.0 | Marzo 2026 | Vercel + Supabase/Postgres | Documento operativo

Este documento deja claro dónde vive cada ambiente, cómo se despliega, cómo se monitorea y cómo se recupera el sistema sin caer en ambigüedades ni rituales de último minuto.

Tema	Decisión	Resultado operativo
Ambientes oficiales	local, preview, production	flujo simple y auditable desde el día uno
Hosting web	Vercel	preview deployments por branch y producción controlada
Base de datos	Supabase / Postgres	aislamiento y backups gestionados
Rollback	web + estrategia de restauración DB	recuperación clara ante errores

1. Tesis del documento

Capataz necesita una columna vertebral de ambientes e infraestructura que permita construir rápido sin volver frágil la operación. La regla es simple: cada ambiente existe para una misión distinta y ningún deploy debería sentirse como tirar una moneda al aire.

2. Principios rectores

- Simplicidad con disciplina: pocos ambientes, pero bien definidos.
- Separación real: local, preview y producción no comparten secretos ni datos reales sin control.
- Preview como campo de pruebas: toda feature relevante debe verse antes de tocar producción.
- Producción protegida: los cambios deben ser intencionales, observables y reversibles.
- Base de datos tratada con respeto: migraciones revisables, reproducibles y recuperables.
- Errores visibles: el sistema no debe fallar en silencio.

3. Ambientes oficiales

3.1 Local

Ambiente de desarrollo individual para Ricardo, Víctor y cualquier integrante técnico autorizado.

- desarrollo diario, debugging y prototipado rápido
- usa `.env.local` privado y datos semilla o mocks
- jamás apunta a producción por default
- puede usar Supabase dev o Postgres local si conviene

3.2 Preview

Ambiente compartible para QA, demos privadas y validación de aceptación.

- Vercel genera deployments por branch o PR
- usa variables de preview y base no productiva
- permite revisar UX, permisos, reglas y score mock
- sirve como antesala obligatoria del merge a `main`

3.3 Production

Ambiente real de operación del negocio.

- usa dominio principal o subdominio oficial
- solo consume secretos productivos
- solo recibe código aprobado y validado
- requiere monitoreo, auditoría y backups activos

4. Modelo operativo por ambiente

Dimensión	Local	Preview	Production
Hosting web	localhost	Vercel Preview Deployment	Vercel Production
Base de datos	Supabase dev o Postgres local	Supabase preview / staging	Supabase production
Datos	semilla, mock, anonimizado	demo, staging sanitizado	reales autorizados
Auth	no productiva	no productiva	productiva
Secrets	`.env.local`	Vercel Preview Env Vars	Vercel Production Env Vars
Riesgo aceptable	alto controlado	medio	bajo

5. Dominio temporal y nomenclatura de URLs

Mientras llega el dominio final del cliente, Capataz puede arrancar con un dominio temporal controlado. La recomendación es usar una URL productiva temporal estable y dejar las previews solo para validación.

- `capataz-dashboard.vercel.app` o `capataz-operativo.vercel.app` como primera capa
- preview deployments automáticos de Vercel para ramas y pull requests
- migración posterior a dominio final como `app.capataz.ai` cuando el cliente lo adquiera
- al migrar, se actualizan callbacks de auth, DNS, CSP y allowlists

6. Variables por ambiente

No se documentan valores reales dentro del repositorio. Solo nombres, propósito y disponibilidad por ambiente.

6.1 Aplicación y runtime

Variable	Local	Preview	Production	Propósito
NODE_ENV	development	production	production	modo de runtime
APP_ENV	local	preview	production	identificador funcional
NEXT_PUBLIC_BASE_URL	sí	sí	sí	URL base pública
INTERNAL_API_BASE_URL	sí	sí	sí	URL interna para jobs y server

6.2 Supabase y base de datos

Variable	Local	Preview	Production	Propósito
NEXT_PUBLIC_SUPABASE_URL	sí	sí	sí	URL pública del proyecto
NEXT_PUBLIC_SUPABASE_ANON_KEY	sí	sí	sí	acceso cliente controlado
SUPABASE_SERVICE_ROLE_KEY	opcional	sí	sí	jobs y tareas servidor
DATABASE_URL	sí	sí	sí	conexión Postgres server-side
DIRECT_DATABASE_URL	opcional	sí	sí	migraciones o tareas directas

6.3 Política de variables

- nunca se suben secretos al repo
- `.env.example` sí se versiona, pero solo con placeholders
- preview y production jamás comparten service role keys
- rotación obligatoria si un secreto se expone
- acceso a variables de producción solo para responsables autorizados

7. Estrategia de despliegue

El flujo recomendado para el MVP es deliberadamente limpio: branch de feature, preview deployment, validación, merge a `main`, producción y smoke test posterior.

- desarrollo en ramas de feature
- preview deployment automático al subir la rama
- QA funcional y revisión visual antes de aprobar

- merge a `main` como única puerta de producción
- smoke test post-deploy en dashboard, login, tareas, kanban y score

Checklist mínimo de release

Control	Criterio
Build	verde
Pruebas críticas	verdes
Preview	aprobado
Migraciones	revisadas
Variables	presentes por ambiente
Backup	disponible antes de cambios riesgosos
Smoke test	definido y ejecutado

8. Rollback y recuperación

Rollback no es un botón mágico; es una secuencia premeditada. La web puede revertirse rápido. La base de datos exige más cabeza fría.

- Frontend / app: redeploy de un build previo estable en Vercel y validación con smoke test.
- Backend lógico: revertir feature flags o desactivar rutas problemáticas si aplica.
- Base de datos: preferir fix forward o script correctivo; restore completo solo en incidentes serios.
- Toda restauración productiva debe evaluar impacto de pérdida de datos antes de ejecutarse.

9. Backups de base de datos

- backups automáticos gestionados por Supabase o proveedor equivalente
- confirmación de retención activa
- respaldo extra manual previo a migraciones críticas
- restauración probada al menos una vez en ambiente no productivo
- preview jamás usa datos reales sin sanitización o permiso explícito

10. Monitoreo y observabilidad

Capa	Señales mínimas
Frontend	errores de render, hidratación, fallos JS críticos, tiempos de carga clave
Backend	errores 5xx, latencia, fallos de jobs, errores de validación relevantes
Base de datos	disponibilidad, conexiones, queries lentas, crecimiento

	anómalo
Auth	logins fallidos, callbacks rotos, expiración anómala de sesiones
Infra web	build failures, deploy failures, health checks, incidentes de dominio
Negocio	score no recalculado, cron fallido, alertas no generadas, tareas atascadas

Herramientas sugeridas

- Vercel Analytics y logs para capa web
- logs estructurados del servidor
- Sentry o equivalente para error tracking
- panel de Supabase para DB y auth
- webhook de alertas internas para incidentes prioritarios

11. Manejo de errores por ambiente

Local

- stacktrace visible
- logs verbosos
- facilidad de debugging por encima del polish

Preview

- errores visibles sin exponer secretos
- tracking activo para validar regresiones
- mensajes claros para QA y demo

Production

- mensajes amigables al usuario
- detalle técnico solo en logs seguros
- `traceId` visible en errores críticos
- fallback controlado en componentes del dashboard

12. Política de acceso operativo

- producción no se usa para experimentar
- se evita editar datos manualmente en prod
- secrets de producción no se comparten por chat abierto
- accesos se revocan al terminar colaboración
- toda intervención sensible queda documentada

13. Topología recomendada del MVP

- Next.js app en Vercel
- Server Actions y API Routes para mutaciones y lecturas protegidas
- Supabase Auth + Postgres principal
- Storage para exportes y adjuntos controlados si aplica
- cron de score diario y reconciliación de alertas protegido con `CRON_SECRET`
- futuro cercano: Meta Cloud API, colas o workers dedicados y un entorno staging separado cuando el negocio lo pida

14. Runbook mínimo del equipo

Evento	Acción inmediata	Responsable primario
Falla en preview	revisar logs de build y runtime, corregir branch y redeploy	desarrollador autor
Falla en producción web	evaluar severidad, rollback de release si aplica y comunicar estado	líder técnico
Error de migración	detener despliegue, revisar impacto y decidir fix forward o restore	responsable de DB
Cron de score fallido	revisar logs, relanzar job y validar snapshots	backend / owner técnico
Error de auth	revisar callbacks, cookies, variables y proveedor	owner de plataforma

15. Definición de terminado

- existe definición clara de local, preview y production
- las variables por ambiente y su propósito están documentadas
- hay dominio temporal definido para arranque
- la estrategia de deploy y rollback está cerrada
- backups y monitoreo tienen política mínima explícita
- el manejo de errores no queda a improvisación

16. Decisión ejecutiva recomendada

Para el MVP de Capataz: web en Vercel, Supabase / Postgres por ambiente no productivo y productivo separados, preview deployments obligatorios, producción desplegada solo desde main, backups activos y datos demo sanitizados.